



North South University

Department of Electrical and Computer Engineering

CSE 215L (Programming Language II Lab)

Lab 3: Conditional Statements and Loops

Objective

- **Conditional Statements Revision:** Review the use of `if`, `if-else`, `else-if`, nested conditions, and `switch` statements for controlling program flow.
- **Loop Structures:** Understand the three main loop structures in Java: `for`, `while`, and `do-while`.
- **Choosing the Appropriate Loop:** Understand the difference between the three loops and identify when each loop is suitable for a particular problem.
- **Loop Control Statements:** Learn to use `break` and `continue` to modify the normal flow of a loop.
- **Problem Solving:** Apply conditional statements and loops to solve basic programming problems using user input and repeated execution.

Conditional Statements – Revision

Conditional statements are used to make decisions based on whether a condition is `true` or `false`. Since conditional statements were covered in Lab 2, this section is intended as a short revision.

if-else

The `if-else` statement provides two possible paths.

```
if (condition) {  
    // Execute when condition is true  
} else {  
    // Execute when condition is false  
}
```

Multiple Conditions

When several conditions need to be checked, `else-if` can be used.

```
if (marks >= 80) {  
    System.out.println("A");  
} else if (marks >= 70) {  
    System.out.println("B");  
} else if (marks >= 60) {  
    System.out.println("C");  
} else {  
    System.out.println("F");  
}
```

Conditions are checked from top to bottom. Once a condition is satisfied, the remaining conditions are skipped.

Nested Conditions

A conditional statement can be placed inside another conditional statement. This is called a nested conditional.

```
if (age >= 18) {  
    if (hasID) {  
        System.out.println("Access granted");  
    } else {  
        System.out.println("ID required");  
    }  
}
```

```

} else {
    System.out.println("Access denied");
}

```

Switch

The **switch** statement is useful when one expression needs to be compared with several fixed values.

```

switch (day) {
    case 1:
        System.out.println("Sunday");
        break;
    case 2:
        System.out.println("Monday");
        break;
    case 3:
        System.out.println("Tuesday");
        break;
    default:
        System.out.println("Invalid day");
}

```

The **break** statement prevents execution from continuing into the next case. The **default** block executes when none of the cases match.

Loops

Loops are used when a block of code needs to be executed repeatedly. Java provides three main loop structures: **for**, **while**, and **do-while**.

The main difference is **when the condition is checked and how the loop is normally controlled**.

Loop	How it works	Typical use
for	The initialization, condition, and update are written together. The condition is checked before each iteration.	When the number of repetitions is known or can be counted.
while	The condition is checked before each iteration. The initialization and update are usually written separately.	When repetition depends on a condition and the number of iterations may not be known beforehand.
do-while	The loop body executes first, and the condition is checked afterward. Therefore, the body runs at least once.	When the statements must execute at least once, such as menu-driven input or validation.

for Loop

A **for** loop places the initialization, condition, and update in one line.

```

for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}

```

Execution order:

1. Initialize **i**.
2. Check the condition.
3. Execute the loop body if the condition is true.
4. Update **i**.
5. Check the condition again.

while Loop

A **while** loop checks its condition before executing the body.

```

int i = 1;

while (i <= 5) {
    System.out.println(i);
}

```

```
i++;
}
```

If the condition is false before the first iteration, the loop body does not execute.

do-while Loop

A **do-while** loop executes its body before checking the condition.

```
int i = 1;

do {
    System.out.println(i);
    i++;
} while (i <= 5);
```

Because the condition is checked after the body, a **do-while** loop always executes at least once.

Key Difference Between the Three Loops

Loop	Condition checked	Runs at least once?
for	Before each iteration	No
while	Before each iteration	No
do-while	After each iteration	Yes

Simple Example

All three loops can produce the same output when used appropriately.

```
for (int i = 1; i <= 3; i++) {
    System.out.println(i);
}

int j = 1;
while (j <= 3) {
    System.out.println(j);
    j++;
}

int k = 1;
do {
    System.out.println(k);
    k++;
} while (k <= 3);
```

The important difference is not what the loops *can* accomplish, but how their conditions and repetition are structured.

Loop Control Statements

Two commonly used statements can change the normal flow of a loop: **break** and **continue**.

break

The **break** statement immediately terminates the loop.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    System.out.println(i);
}
```

The output is:

```
1
2
3
4
```

continue

The **continue** statement skips the current iteration and moves to the next iteration.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    System.out.println(i);
}
```

The output is:

```
1
2
4
5
```

Classwork

Complete the following tasks.

1. **(Using if-else)** Take a user input for your name and check whether it starts with an uppercase vowel. If the condition is fulfilled, write “My name starts with a vowel”, else write “My name starts with a consonant”.
2. **(Using while loop)** Take an integer **n** through user input and print the summation of all numbers from zero to **n**.
3. **(Convert for loop)** Consider the following program. Rewrite the program using a **while** loop without changing its output.

```
int sum = 0;

for (int i = 1; i <= 20; i++) {
    if (i % 2 == 0) {
        sum += i;
    } else {
        System.out.println("Skipping: " + i);
    }
}

System.out.println("Sum of even numbers: " + sum);
```

Homework

Complete the following tasks.

1. Write a program that takes an integer and determines if it is odd or even. Use **switch** cases to produce the result.
2. Print a pattern with plus signs and minus signs alternating with each other. An example is given below:

```
+
+ -
+ - +
+ - + -
+ - + - +
```

3. Create a program where three integers are taken as user input, followed by performing one of the following functions:
 - (a) Press ‘1’ for addition of numbers.
 - (b) Press ‘2’ for the subtraction of the other numbers from the maximum.
 - (c) Press ‘3’ for multiplication of numbers.
 - (d) Any other input will give a warning for “Invalid Input”.